

Website Analytics API - Node.js backend using Express, PostgreSQL, Redis, and Docker.

Implements: API Key Management, Event Collection, Analytics Endpoints, Rate Limiting, Caching, Testing, Swagger Docs.

Folder structure: src/{index.js, db.js, routes/{auth.js, analytics.js}, controllers/{authController.js, analyticsController.js}, models/{apiKeyModel.js, eventModel.js}, middleware/{apiKeyAuth.js, rateLimiter.js}, utils/cache.js, tests/{auth.test.js, analytics.test.js}}, Dockerfile, docker-compose.yml, docker-init.sql, swagger.js, .env.example.

package.json includes Express, pg, redis, swagger-ui-express, jest, supertest, express-rate-limit.
API Endpoints: /api/auth/register (POST) generate API key, /api/auth/api-key (GET) retrieve info, /api/auth/revoke (POST) revoke key, /api/auth/regenerate (POST) new key, /api/analytics/collect (POST) submit events, /api/analytics/event-summary (GET) fetch summary, /api/analytics/user-stats (GET) user stats.

Database Schema: api_keys(id, key, name, created_at, expires_at, revoked), events(id, api_key, app_id, user_id, event_type, url, referrer, device, metadata, created_at).

Environment Variables: DATABASE_URL, REDIS_URL, PORT, API_KEY_TTL_DAYS, REQUESTS_PER_MINUTE.

Run locally: docker-compose up --build, Swagger Docs: http://localhost:3000/docs.

Deploy easily on Render or Railway with Docker support, add environment variables.

Testing: npm test using Jest and Supertest, checks API endpoints.

Judging Criteria: Code quality, scalability, completeness, deployment readiness, and functional endpoints.